

Mikroişlemci Sistemleri

Proje Raporu

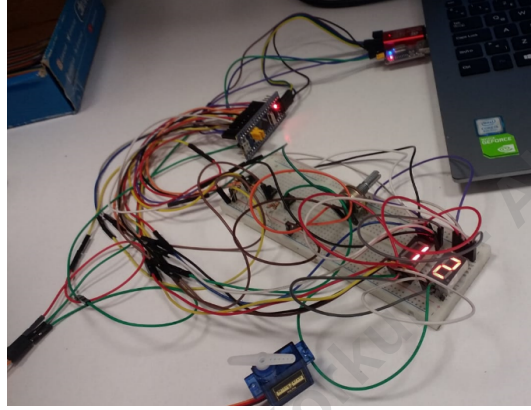
Grup : Perşembe /Saat 14 grubu

Ad-Soyad : Muhammed Emin KORKUNÇ -2021221054

Abdullah Sina Korkmaz -2021221044

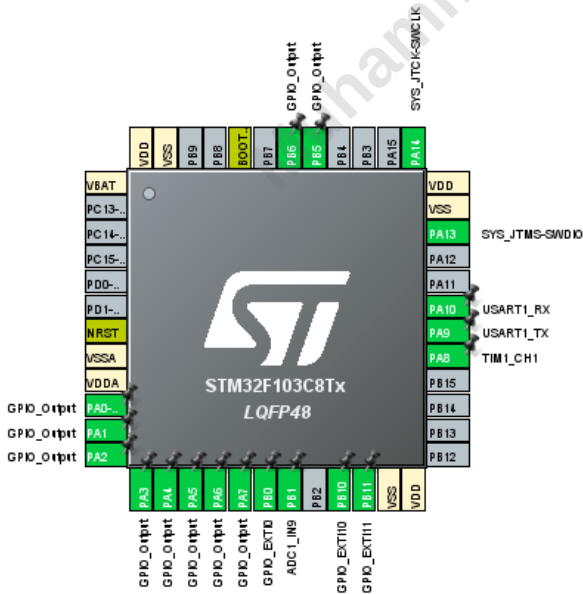
Bölüm : Bilgisayar Mühendisliği 2.sınıf / Fatih Sultan Mehmet Vakıf Üniversitesi

Projenin bitmiş halinin görseli:



Şekil 1 Proje bitmiş halinin görseli

Proje sırasında kullanılan çevre birimleri ayarları:



Şekil 2 CubeMX pin seçimleri

GPIO A0-A7 pinleri output olarak kullanılmış ve 7-segment'in ilgili pinlerine bağlanmıştır.

B0-B10-B11 pinleri 7-segmenti kontrol eden butonlara (arttırma-azaltma-reset) bağlanmaktadır. External interrupt olarak kullanılmıştır. PB6 ve PB5 pinleri 1.ve 2. 7-Segmentin ground pinlerine ayrı ayrı bağlanmış ve onlar -birler basamağını kontrol etmeyi sağlamıştır.

While bloğu içinde ki buton arttırma azaltma ve resetleme işlemlerinden gelen sayı onlar ve birler basamağına ayrılır değeri USER CODE BEGIN 0 içinde değeri yaz fonksiyonu ile 7-Segmente gidecek şekilde çevrilir. Interrupt içine yazdığımız sayı arttırma azaltma resetlem işlemleri ile butonu anında kontrol edebilir ve butona basıldığında istenilen işi direkt yapmasını sağlarız. Debug da aldığımız değerleri gösterdik.

```

void EXTI0_IRQHandler(void)
{
    /* USER CODE BEGIN EXTI0_IRQn 0 */

    sayi++;

    /* USER CODE END EXTI0_IRQn 0 */
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);
    /* USER CODE BEGIN EXTI0_IRQn 1 */

    /* USER CODE END EXTI0_IRQn 1 */
}

```

```

void EXTI15_10_IRQHandler(void)
{
    /* USER CODE BEGIN EXTI15_10_IRQn 0 */

    sayi--;

    /* USER CODE END EXTI15_10_IRQn 0 */
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_10);
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_11);
    /* USER CODE BEGIN EXTI15_10_IRQn 1 */

```

```

84 void deger_yaz(int s){
85     switch(s){
86     case 0:
87         GPIOA->ODR = 0x3f;
88         break;
89     case 1:
90         GPIOA->ODR = 0x06;
91         break;
92     case 2:
93         GPIOA->ODR = 0x5b;
94         break;
95     case 3:
96         GPIOA->ODR = 0x4f;
97         break;
98     case 4:
99         GPIOA->ODR = 0x66;
100        break;
101     case 5:
102         GPIOA->ODR = 0x6d;
103        break;
104     case 6:
105         GPIOA->ODR = 0x7d;
106        break;
107     case 7:
108         GPIOA->ODR = 0x07;
109        break;
110     case 8:
111         GPIOA->ODR = 0x7f;
112        break;
113     case 9:
114         GPIOA->ODR = 0x6f;
115        break;
116    }
117 }

```

```

if (HAL_GPIO_ReadPin(GPIOB,GPIO_PIN_0)==GPIO_PIN_SET)
{
    // B0 butonuna basildiginda
    sayi++;
    HAL_Delay(100);
}
else if (HAL_GPIO_ReadPin(GPIOB,GPIO_PIN_11)==GPIO_PIN_SET)
{
    // B10 butonuna basildiginda
    sayi--;
    HAL_Delay(100);
}
else if (HAL_GPIO_ReadPin(GPIOB,GPIO_PIN_10)==GPIO_PIN_SET)
{
    // B11 butonuna basildiginda
    sayi = 0;
    HAL_Delay(100);
}

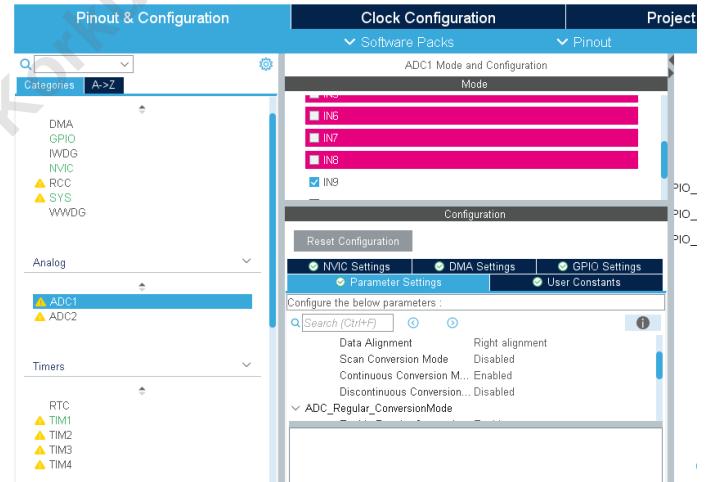
HAL_GPIO_WritePin(GPIOB,GPIO_PIN_5,GPIO_PIN_RESET);
HAL_GPIO_WritePin(GPIOB,GPIO_PIN_6,GPIO_PIN_SET);
deger_yaz(sayi % 10);

HAL_Delay(10);
HAL_GPIO_WritePin(GPIOB,GPIO_PIN_5,GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOB,GPIO_PIN_6,GPIO_PIN_RESET);
deger_yaz(sayi / 10);

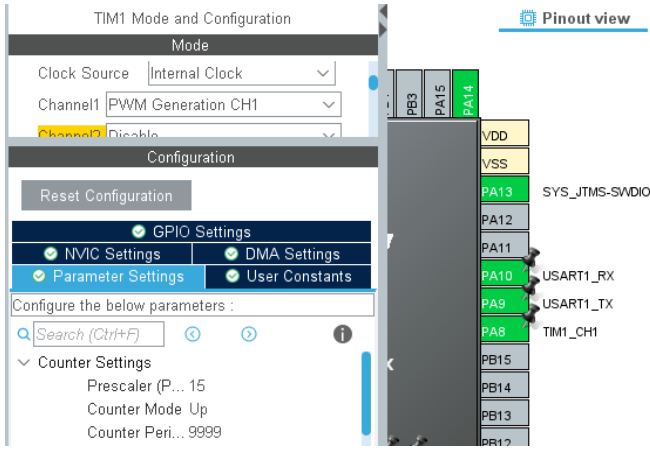
HAL_Delay(10);
if(sayi >= 100){
    sayi = 1;
}

```

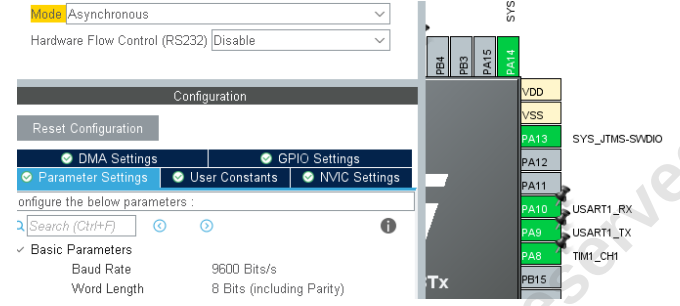
B1 pini ADC olarak kullanılmış potansiyometreye bağlanmıştır .Continuous Conversion enable edilmiştir.



PA8 pini için TIM1_CH1 seçilmiş ve servo motora bağlanmıştır ve servo motor 5V ve ground kablolarıyla bağlanmıştır.



Seri Haberleşme USART1



Frekans=Saat kaynak frekansı/ (PSC+1)(ARR+1)

Servo motor için 50 Hz sinyale ihtiyacım vardır.Saat frekansı = 8 000 000 Hz . Prescaler 16 ile bölersem. Frekans = 8000000/16 = 500000Hz. Sayaç periyoduna tekrar bölersem 10000. Frekans = 500000/10000 = 50Hz. Sonuç olarak Prescaler değeri 15, ARR değeri 9999 olarak ayarlanmıştır.

ADC den gelen değer 0-4095 arasında olduğundan bunu servo motorun kullanabileceği 0-1250 değerleri arasına çevirmemiz gerekir. Bunun için aşağıdaki kod yazılmıştır. PWM = 250 + readValue / 4.1; ve bu işlem yapılmıştır.

```
199 HAL_ADC_PollForConversion(&hadc1, 1000);
200 readValue = HAL_ADC_GetValue(&hadc1);
201 PWM = 250 + readValue / 4.1;
202 __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, PWM);
203 HAL_Delay(5);
```

PA10 pini USART1_RX ve PA9 pini USART1_TX olarak ayarladık ve stm32 de A10 pinine TX, PA9 pinine RX bağlanmıştır

```
while (1)
{

    int angle = calculateAngleFromADC();
    char degr[20];
    sprintf(degr, "the degree is:%d", angle);
    sendToTermite(degr);
}
```

Baud Rate değeri saniye gönderilen verinin hızını temsil eder. Hız = 9600/(start bit + Data + Stop bit) formülü ile hesaplanır. Biz USART1 modunu Asynchronous olarak ayarladık. Baud Rate değerini 9600 yaptık. Global interrupt' ı etkinleştirdik.

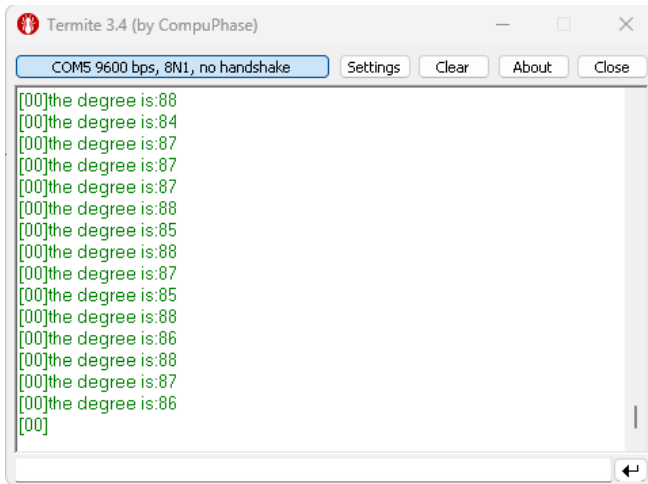
Aşağıda ki kodlarda seri haberleşme için yaptıklarımız mevcuttur. Potansiyometreden aldığımız PWM değerini açı değerine çevirdik ve arasında servo motorun hareketlerini potansiyometre ile kontrol ederek açı değışimerilni Termite de 0-180 derece arasında gözlemledik. Böylece seri haberleme kısmını da tamamladık.

```
120 float calculateAngleFromADC(void) {
121     HAL_ADC_Start(&hadc1);
122
123     if (HAL_ADC_PollForConversion(&hadc1, 1000) == HAL_OK) { //
124         uint32_t adcValue = HAL_ADC_GetValue(&hadc1);
125         servo = ((adcValue - 0) * (1750 - 550)) / (4096 - 0) + 0;;
126         float angle = ((adcValue-0)*(180-0))/(4096-0)+0;
127         return angle;
128     }
129     return 0.0f; // Dönüşüm basarisiz oldugunda 0.0 döndür
130 }
```

```

76 void sendToTermite(char* data) {
77
78     size_t size = strlen(data) + 3;
79     char modifiedData[size];
80     strcpy(modifiedData, data);
81     strcat(modifiedData, "\r\n");
82     HAL_UART_Transmit(&uart1, (uint8_t*)modifiedData, size, 1000);
83 }

```



```

/* USER CODE BEGIN PV */
int sayi;
int servo;

#include <stdio.h>
#include "string.h"
uint8_t tData[] = "dx\r\n";
uint8_t getData[20];

```

Muhammed Emin Korkunc - All Rights Reserved